# Reinforcement Learning: Setting Up Gymnasium Environments

*Lab 1 Task Report*

**Submitted by:**
Junaid Asif

**Roll No:** BSAI-144

**Instructor:**
Ms. Hina Ali

February 20, 2026

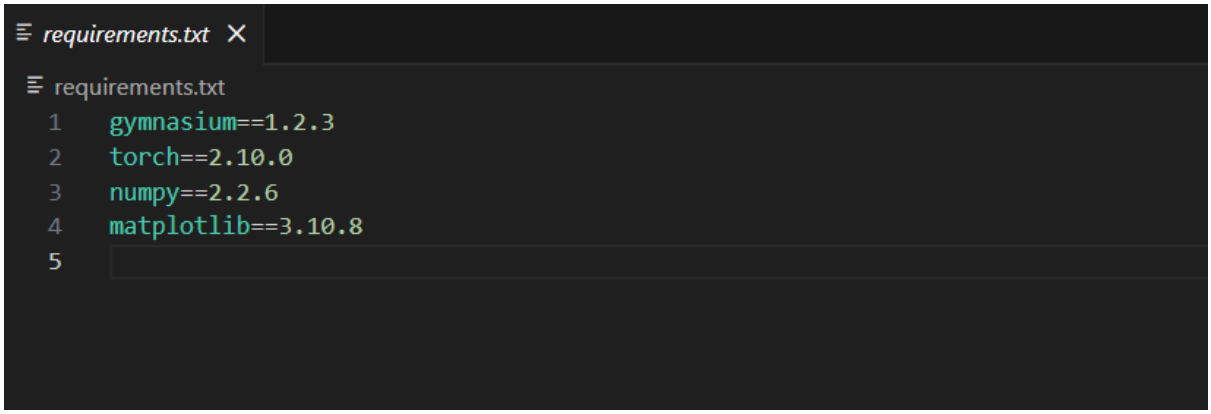


Figure 1: requirements.txt with pinned library versions

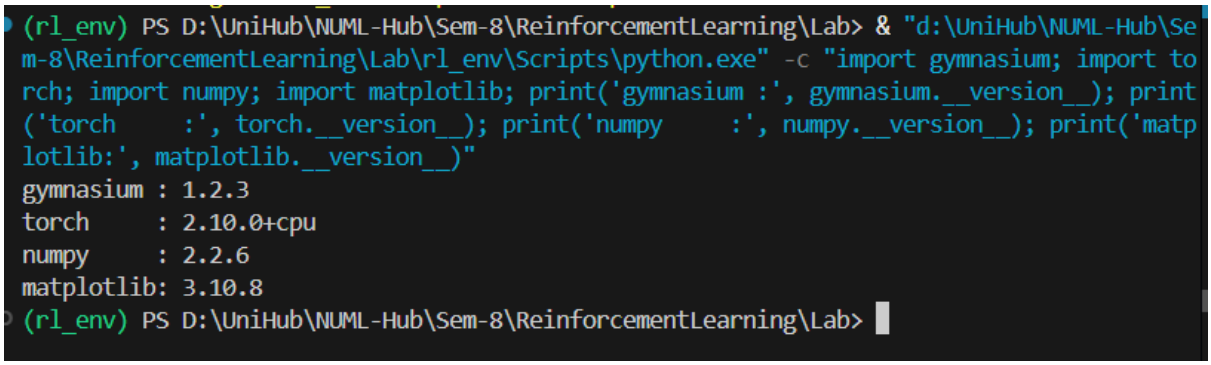


Figure 2: Installed packages verification (pip list)

env1_cartpole.py

```python
# Environment 1: CartPole-v1
# --------------------------

# Step 1: Create the environment (render_mode="human" opens a visual window)
env = gym.make("CartPole-v1", render_mode="human")

# Step 2: Print basic environment info
print("Environment Name  :", "CartPole-v1")
print("Observation Space :", env.observation_space)   # what the agent sees
print("Action Space      :", env.action_space)        # 0 = push left, 1 = push right

# Step 3: Reset the environment to get starting state
observation, info = env.reset()

print("\nStarting Observation:", observation)
print("  Cart Position :", observation[0])
print("  Cart Velocity :", observation[1])
print("  Pole Angle    :", observation[2])
print("  Pole Velocity :", observation[3])
print("Info            :", info)

# Step 4: Run 5 steps with random actions
print("\n--- Running 50 Steps ---")

for step in range(50):

    # Pick a random action
    action = env.action_space.sample()

    # Take the action
    observation, reward, terminated, truncated, info = env.step(action)

    # Is the episode over?
    done = terminated or truncated

    # Print all variables
    print(f"\nStep      : {step + 1}")
    print(f"Action    : {action}  (0=Left, 1=Right)")
    print(f"Obs[0] Cart Position : {observation[0]:.4f}")
    print(f"Obs[1] Cart Velocity : {observation[1]:.4f}")
    print(f"Obs[2] Pole Angle    : {observation[2]:.4f}")
    print(f"Obs[3] Pole Velocity : {observation[3]:.4f}")
    print(f"Reward    : {reward}")
    print(f"Terminated: {terminated}")
    print(f"Truncated : {truncated}")
    print(f"Done      : {done}")
    print(f"Info      : {info}")

    if done:
        observation, info = env.reset()

# Step 5: Close the environment
env.close()
print("\nDone!")
```

Figure 3: CartPole-v1 – Python code

```
(rl_env) PS D:\UniHub\NUML-Hub\Sem-8\ReinforcementLearning\Lab> & D:/UniHub/NUML-Hub/Sem-8/ReinforcementLearning/Lab/rl_env/Scripts/python.exe d:/UniHub/NUML-Hub/Sem-8/ReinforcementLearning
/Lab/env1_cartpole.py
Environment Name  : CartPole-v1
Observation Space : Box([-4.8               -inf -0.41887903        -inf], [4.8               inf 0.41887903        inf], (4,), float32)
Action Space      : Discrete(2)

Starting Observation: [ 0.00113329  0.0417241  -0.01501006 -0.02944009]
  Cart Position : 0.0011332886
  Cart Velocity : 0.041724104
  Pole Angle    : -0.015010055
  Pole Velocity : -0.029440088
Info            : {}

--- Running 50 Steps ---

Step      : 1
Action    : 1  (0=Left, 1=Right)
Obs[0] Cart Position : 0.0020
Obs[1] Cart Velocity : 0.2371
Obs[2] Pole Angle    : -0.0156
Obs[3] Pole Velocity : -0.3268
Reward    : 1.0
Terminated: False
Truncated : False
Done      : False
Info      : {}
```

Figure 4: CartPole-v1 – Terminal output showing all variable values

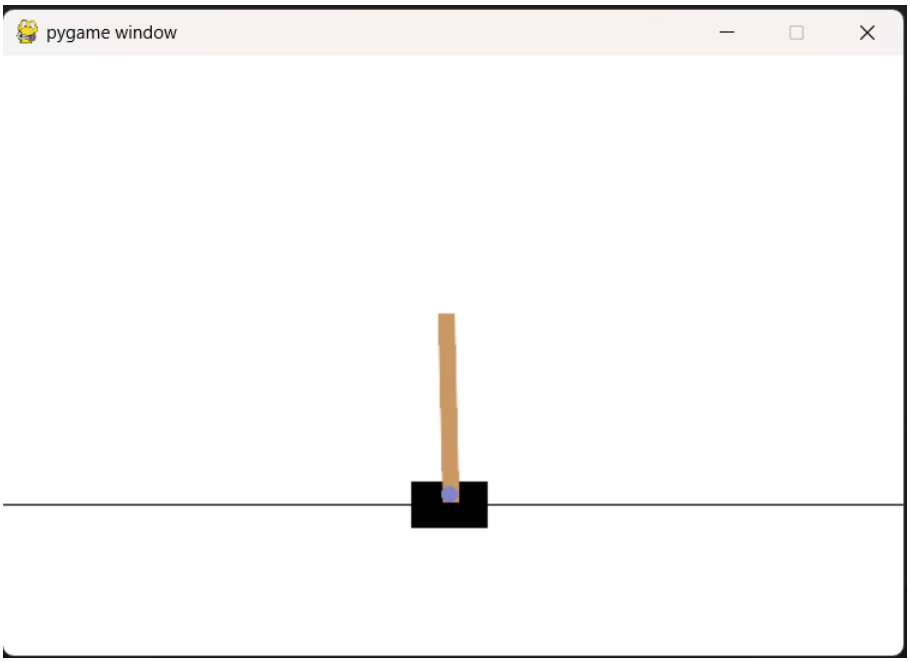


Figure 5: CartPole-v1 – Visual render window

env2_frozenlake.py > ...

```python
import gymnasium as gym

# ----------------------------
# Environment 2: FrozenLake-v1
# ----------------------------

# Step 1: Create the environment (render_mode="human" opens a visual window)
env = gym.make("FrozenLake-v1", is_slippery=False, render_mode="human")

# Step 2: Print basic environment info
print("Environment Name  :", "FrozenLake-v1")
print("Observation Space :", env.observation_space)   # 16 tiles (4x4 grid)
print("Action Space      :", env.action_space)         # 0=Left 1=Down 2=Right 3=Up
print("Number of States  :", env.observation_space.n)
print("Number of Actions :", env.action_space.n)

# Step 3: Reset the environment
observation, info = env.reset()

print("\nStarting Observation (tile number):", observation)
print("Info :", info)
print("  prob = probability of this transition:", info.get("prob", "N/A"))

# Step 4: Run 5 steps with random actions
print("\n--- Running 50 Steps ---")

action_names = {0: "LEFT", 1: "DOWN", 2: "RIGHT", 3: "UP"}

for step in range(50):

    # Pick a random action
    action = env.action_space.sample()

    # Take the action
    observation, reward, terminated, truncated, info = env.step(action)

    # Is the episode over?
    done = terminated or truncated

    # Print all variables
    print(f"\nStep        : {step + 1}")
    print(f"Action      : {action}  ({action_names[action]})")
    print(f"Observation : {observation}  (current tile on grid)")
    print(f"Reward      : {reward}  (1 = reached goal, 0 = otherwise)")
    print(f"Terminated  : {terminated}")
    print(f"Truncated   : {truncated}")
    print(f"Done        : {done}")
    print(f"Info        : {info}")

    if done:
        print("Episode ended. Resetting...")
        observation, info = env.reset()

# Step 5: Close the environment
env.close()
print("\nDone!")

```

Figure 6: FrozenLake-v1 – Python code

```
(rl_env) PS D:\UniHub\NUML-Hub\Sem-8\ReinforcementLearning\Lab> & D:/UniHub/NUML-Hub/Sem-8/ReinforcementLearning/Lab/rl_env/Scripts/python.exe d:/UniHub/NUML-Hub/Sem-8/ReinforcementLearning
/Lab/env2_frozenlake.py
Environment Name  : FrozenLake-v1
Observation Space : Discrete(16)
Action Space      : Discrete(4)
Number of States  : 16
Number of Actions : 4

Starting Observation (tile number): 0
Info : {'prob': 1}
  prob = probability of this transition: 1

--- Running 50 Steps ---

Step        : 1
Action      : 1  (DOWN)
Observation : 4  (current tile on grid)
Reward      : 0  (1 = reached goal, 0 = otherwise)
Terminated  : False
Truncated   : False
Done        : False
Info        : {'prob': 1.0}

Step        : 2
Action      : 0  (LEFT)
Observation : 4  (current tile on grid)
Reward      : 0  (1 = reached goal, 0 = otherwise)
Terminated  : False
Truncated   : False
Done        : False
Info        : {'prob': 1.0}

Step        : 3
Action      : 0  (LEFT)
Observation : 4  (current tile on grid)
Reward      : 0  (1 = reached goal, 0 = otherwise)
Terminated  : False
Truncated   : False
Done        : False
Info        : {'prob': 1.0}
```

Figure 7: FrozenLake-v1 – Terminal output showing all variable values


Figure 8: FrozenLake-v1 – Visual render showing the 4x4 grid

env3_mountaincar.py > ...

```python
import gymnasium as gym

# ----------------------------
# Environment 3: MountainCar-v0
# ----------------------------

# Step 1: Create the environment (render_mode="human" opens a visual window)
env = gym.make("MountainCar-v0", render_mode="human")

# Step 2: Print basic environment info
print("Environment Name  :", "MountainCar-v0")
print("Observation Space :", env.observation_space)   # position and velocity
print("Action Space      :", env.action_space)         # 0=Left 1=Nothing 2=Right
print("Position Range    : min =", env.observation_space.low[0],  "max =", env.observation_space.high[0])
print("Velocity Range    : min =", env.observation_space.low[1],  "max =", env.observation_space.high[1])

# Step 3: Reset the environment
observation, info = env.reset()

print("\nStarting Observation:", observation)
print("  Position :", observation[0])
print("  Velocity :", observation[1])
print("Info       :", info)

# Step 4: Run 5 steps with random actions
print("\n--- Running 50 Steps ---")

action_names = {0: "PUSH LEFT", 1: "NO PUSH", 2: "PUSH RIGHT"}

for step in range(50):

    # Pick a random action
    action = env.action_space.sample()

    # Take the action
    observation, reward, terminated, truncated, info = env.step(action)

    # Is the episode over?
    done = terminated or truncated

    # Print all variables
    print(f"\nStep       : {step + 1}")
    print(f"Action     : {action}  ({action_names[action]})")
    print(f"Position   : {observation[0]:.4f}  (car position on the hill)")
    print(f"Velocity   : {observation[1]:.4f}  (car speed)")
    print(f"Reward     : {reward}  (-1 every step until goal reached)")
    print(f"Terminated : {terminated}")
    print(f"Truncated  : {truncated}")
    print(f"Done       : {done}")
    print(f"Info       : {info}")

    if done:
        print("Episode ended. Resetting...")
        observation, info = env.reset()

# Step 5: Close the environment
env.close()
print("\nDone!")
```

Figure 9: MountainCar-v0 – Python code

```
(rl_env) PS D:\UniHub\NUML-Hub\Sem-8\ReinforcementLearning\Lab> & D:/UniHub/NUML-Hub/Sem-8/ReinforcementLearning/Lab/rl_env/Scripts/python.exe d:/UniHub/NUML-Hub/Sem-8/ReinforcementLearning
/Lab/env3_mountaincar.py
Environment Name  : MountainCar-v0
Observation Space : Box([-1.2  -0.07], [0.6  0.07], (2,), float32)
Action Space      : Discrete(3)
Position Range    : min = -1.2 max = 0.6
Velocity Range    : min = -0.07 max = 0.07

Starting Observation: [-0.47174078  0.        ]
  Position : -0.47174078
  Velocity : 0.0
Info       : {}

--- Running 50 Steps ---

Step       : 1
Action     : 1  (NO PUSH)
Position   : -0.4721  (car position on the hill)
Velocity   : -0.0004  (car speed)
Reward     : -1.0  (-1 every step until goal reached)
Terminated : False
Truncated  : False
Done       : False
Info       : {}

Step       : 2
Action     : 2  (PUSH RIGHT)
Position   : -0.4719  (car position on the hill)
Velocity   : 0.0002  (car speed)
Reward     : -1.0  (-1 every step until goal reached)
Terminated : False
Truncated  : False
Done       : False
Info       : {}

Step       : 3
Action     : 2  (PUSH RIGHT)
Position   : -0.4711  (car position on the hill)
Velocity   : 0.0008  (car speed)
Reward     : -1.0  (-1 every step until goal reached)
Terminated : False
Truncated  : False
Done       : False
Info       : {}
```

Figure 10: MountainCar-v0 – Terminal output showing all variable values

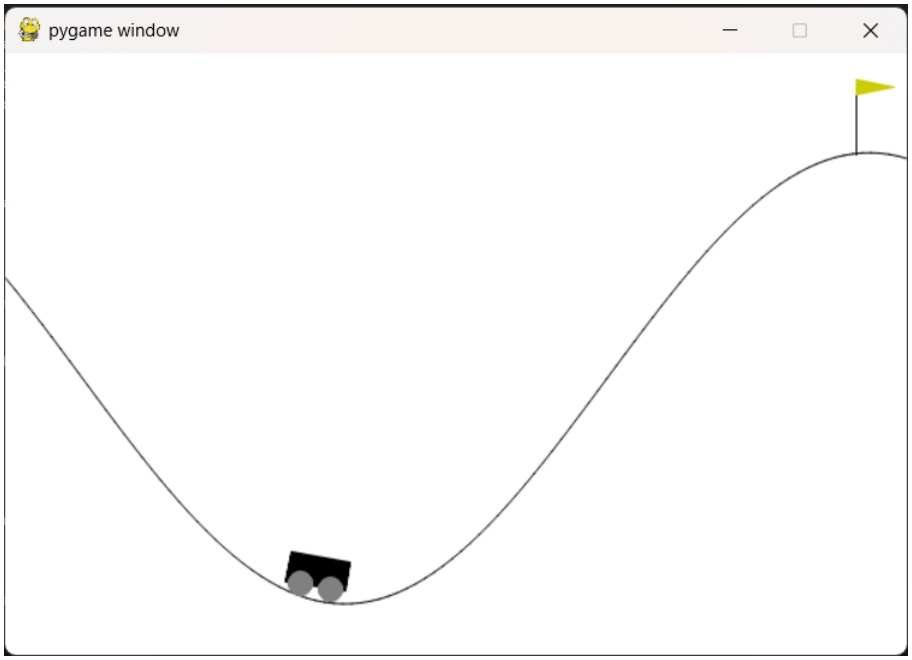


Figure 11: MountainCar-v0 – Visual render showing the car on the hill